ACTIVIDAD PRÁCTICA – CIFRADO CÉSAR, CSV Y JDBC

2.º DAM · Acceso a Datos

1. Planteamiento

En esta actividad se desarrollará una aplicación Java que lea mensajes de un archivo CSV, los cifre mediante el cifrado César y almacene en una base de datos tanto el texto original como el texto cifrado. Finalmente, la aplicación consultará la tabla y mostrará sus registros por pantalla.

El objetivo es practicar el procesamiento de ficheros de texto, la organización del programa en métodos, la transformación de cadenas y el acceso a una base de datos relacional mediante JDBC.

2. Funcionamiento del cifrado César

El cifrado César sustituye cada letra del mensaje por otra situada un número determinado de posiciones más adelante en el alfabeto. El número de posiciones será el desplazamiento indicado para cada mensaje.

```
letra_cifrada = letra_original desplazada N posiciones
```

Se utilizará el alfabeto inglés de la A a la Z. Las letras mayúsculas y minúsculas deberán conservar su formato. Los espacios, números y signos de puntuación no se modificarán. Las vocales acentuadas y la letra ñ, al no pertenecer al alfabeto utilizado, permanecerán sin cambios.

El desplazamiento deberá estar comprendido entre 1 y 25. El cifrado deberá aplicar el retorno al principio del alfabeto cuando se supere la letra Z.

Ejemplo con un desplazamiento de 3:

```
HOLA MUNDO  →  KROD PXQGR
```

3. Fichero de entrada CSV

La aplicación leerá los mensajes desde un fichero de texto llamado mensajes.csv. Cada línea contendrá dos campos separados por punto y coma (;), sin una línea de encabezado:

```
desplazamiento;texto
```

El primer campo será el desplazamiento numérico y el segundo, el texto que se desea cifrar. El fichero deberá guardarse con una codificación de texto conocida, preferiblemente UTF-8.

Ejemplo de contenido de mensajes.csv:

```
3;HOLA MUNDO
5;ACCESO A DATOS
1;Java es un lenguaje de programacion
13;ATAQUE AL AMANECER
25;ZORRO ROJO
```

El programa leerá el fichero línea por línea, separará sus campos y comprobará que cada registro contiene los datos esperados. Si una línea no tiene el formato correcto o el desplazamiento no está entre 1 y 25, deberá tratar el error de forma controlada e informar de la incidencia.

4. Cifrado de los mensajes

Se implementará un método que reciba el texto original y el desplazamiento y devuelva el texto cifrado. La lógica del cifrado deberá estar separada del código encargado de leer el CSV y del código de acceso a la base de datos.

El método deberá:

- Recorrer los caracteres del texto original.
- Identificar si cada carácter es una letra mayúscula o minúscula del alfabeto A-Z.
- Aplicar el desplazamiento correspondiente, teniendo en cuenta el retorno circular al inicio del alfabeto.
- Mantener sin cambios los espacios, números, signos de puntuación y caracteres que no pertenezcan al alfabeto utilizado.
- Devolver una nueva cadena con el resultado del cifrado.

5. Tabla de la base de datos

La aplicación almacenará los datos en una tabla llamada mensajes_cesar. La tabla tendrá exactamente tres columnas:

- desplazamiento: número de posiciones utilizadas para cifrar el mensaje.
- texto_normal: mensaje original leído del fichero CSV.
- texto_cifrado: resultado de aplicar el cifrado César.

La estructura de la tabla será la siguiente:

```
CREATE TABLE IF NOT EXISTS mensajes_cesar (
    desplazamiento INT NOT NULL,
    texto_normal VARCHAR(1000) NOT NULL,
    texto_cifrado VARCHAR(1000) NOT NULL
);
```

La tabla se creará desde la aplicación Java mediante JDBC. Se presupone que el servidor de base de datos está instalado y que se dispone de una base de datos, un usuario y una contraseña válidos para conectarse.

6. Conexión a la base de datos mediante JDBC

El programa utilizará JDBC para conectarse a la base de datos relacional. El proyecto deberá incluir el controlador JDBC correspondiente al gestor utilizado. Si se trabaja con MySQL y Maven, se incorporará la dependencia com.mysql:mysql-connector-j.

La conexión se obtendrá mediante DriverManager.getConnection(...). Los recursos JDBC, como Connection, Statement, PreparedStatement y ResultSet, deberán cerrarse correctamente utilizando try-with-resources cuando corresponda.

La aplicación deberá gestionar los posibles errores de conexión o de ejecución de sentencias, evitando que los recursos queden abiertos.

7. Carga de datos en la tabla

Una vez establecida la conexión, la aplicación ejecutará las siguientes operaciones:

1. Crear la tabla mensajes_cesar si todavía no existe.
2. Eliminar los registros anteriores de la tabla, sin eliminar la propia tabla, para que las ejecuciones sucesivas no dupliquen los datos.
3. Abrir mensajes.csv y leer sus líneas una a una.
4. Validar cada línea y obtener el desplazamiento y el texto original.
5. Cifrar el texto mediante el método implementado.
6. Insertar el desplazamiento, el texto original y el texto cifrado en mensajes_cesar.

Para insertar los registros se utilizará PreparedStatement y parámetros con ?. La ejecución de las sentencias de creación, borrado e inserción se realizará mediante executeUpdate(). No se deberá construir la sentencia SQL concatenando directamente los valores del texto.

8. Consulta y visualización de los registros

Al terminar la carga, la aplicación realizará una consulta SQL para recuperar los registros almacenados en mensajes_cesar. Para ello utilizará executeQuery() y recorrerá el ResultSet con el método next().

Se mostrarán por pantalla las tres columnas de cada registro: desplazamiento, texto original y texto cifrado. La salida deberá ser clara y permitir comprobar que los datos leídos del CSV se han cifrado y guardado correctamente.

```
DESPLAZAMIENTO | TEXTO ORIGINAL                        | TEXTO CIFRADO
3              | HOLA MUNDO                           | KROD PXQGR
5              | ACCESO A DATOS                       | FHHJXT F IFYTX
1              | Java es un lenguaje de programacion  | Kbwb ft vo mfohvbkf ef qsphsbnbdjpo
13             | ATAQUE AL AMANECER                   | NGNDHR NY NZNARPRE
25             | ZORRO ROJO                           | YNQQN QNIN
```

La consulta deberá obtener los datos de la tabla, no limitarse a mostrar las cadenas que el programa conserva en memoria.

9. Organización del programa y recursos

El código deberá estar organizado en métodos con responsabilidades diferenciadas. Como mínimo, se recomienda separar la lógica de cifrado, la lectura y validación del CSV, la creación y carga de la tabla, y la consulta y visualización de los resultados.

Para el acceso al fichero se podrán utilizar las clases de java.nio.file, como Path, Paths y Files, o las clases de entrada/salida que resulten adecuadas. Se deberá especificar la codificación al leer el archivo y cerrar correctamente los recursos.

El código deberá incluir nombres significativos, comentarios cuando ayuden a comprender la solución y documentación **Javadoc** en las clases y métodos públicos.

10. Comprobaciones y tratamiento de errores

El programa deberá contemplar, al menos, las siguientes situaciones:

- El fichero mensajes.csv no existe o no se puede leer.
- Una línea del CSV no contiene exactamente los dos campos esperados.
- El desplazamiento no es un número entero o está fuera del intervalo de 1 a 25 (se saltará la línea y por *system.out* sacará un mensaje de aviso).
- El texto está vacío o supera la longitud admitida por la columna de la tabla.
- No es posible establecer la conexión con la base de datos por una configuración incorrecta o porque el servidor no está disponible.
- Se produce un error al crear la tabla, borrar los registros, insertar datos o ejecutar la consulta.
- La aplicación se ejecuta varias veces y no duplica los registros anteriores.

Los errores deberán comunicarse con mensajes comprensibles. No se deberán ignorar las excepciones sin informar de lo ocurrido.

11. Pruebas que se deberán realizar

Se comprobará el funcionamiento de la aplicación con mensajes que incluyan mayúsculas, minúsculas, espacios, números y signos de puntuación. También se probarán desplazamientos distintos, incluido el valor 25, y mensajes que contengan caracteres como ñ o vocales acentuadas.

Además, se realizarán pruebas con una línea CSV mal formada, un desplazamiento no válido, un fichero inexistente y una conexión a la base de datos incorrecta. Se comprobará que la consulta final muestra los registros realmente almacenados y que una segunda ejecución no genera duplicados.

12. Entrega

La entrega deberá incluir los siguientes elementos:

- Proyecto Java completo y ejecutable.
- Fichero mensajes.csv utilizado para las pruebas.
- Captura o transcripción de la salida por consola que muestre los registros consultados.
- Breve informe con las pruebas realizadas y los errores controlados.

La solución deberá demostrar la integración del procesamiento de un fichero CSV, la aplicación del cifrado César y el uso de JDBC para crear, insertar y consultar datos en una base de datos relacional.